

App de Análise e Preparação de Ordens para IBKR

Resumo em uma frase

Vamos construir uma app local que analisa mercados, testa estratégias no passado, mostra sinais explicados, prepara ordens com travões de risco e só envia uma ordem para a IBKR quando o utilizador confirma manualmente. Primeiro tudo em simulação/paper trading.

Este documento explica o projeto em linguagem simples. A ideia é perceber o que a app propõe, quais são os objetivos, como chega lá passo a passo, que peças de software entram no sistema e com que sistemas externos a app comunica.

Nota importante: isto não é aconselhamento financeiro. Uma app bem feita pode ajudar a estudar, organizar e disciplinar decisões, mas não elimina o risco de mercado. Day trading é uma atividade de risco elevado.

Índice

- 1. Visão geral: o que a app propõe
- 2. Objetivos e não-objetivos
- 3. Conceitos base explicados sem jargão
- 4. Tecnologia: o que faz cada peça de software
- 5. Integração: a que sistemas a app se liga
- 6. Dados e PostgreSQL: estrutura recomendada
- 7. Segurança, risco e auditoria
- 8. Plano de construção por fases
- 9. Fluxos práticos de utilização
- Anexo - frase de orientação para a equipa

1. Visão geral: o que a app propõe

A app é uma mesa de análise para trading. Não deve ser pensada como um robô que compra e vende sozinho. Deve ser pensada como um copiloto: observa dados, calcula indicadores, identifica sinais, explica o motivo, prepara uma ordem se o utilizador quiser e exige confirmação manual antes de qualquer envio.

A regra de ouro

A app informa e prepara. Quem carrega no gatilho é sempre o utilizador.

O que acontece na prática

1. A app recebe dados de mercado: preços, volume e velas.
2. Calcula indicadores técnicos como médias móveis, RSI, MACD, ATR e VWAP.
3. As estratégias analisam esses indicadores e geram sinais.
4. O dashboard mostra os sinais com uma explicação simples.
5. O utilizador pode ignorar, estudar melhor ou preparar uma ordem.
6. Antes de mostrar a ordem como confirmável, o RiskEngine valida os limites de risco.
7. A ordem só é transmitida para a corretora depois de confirmação manual.
8. Todas as decisões, sinais, ordens e resultados ficam registados.

2. Objetivos e não-objetivos

Objetivos

- Analisar ativos financeiros de forma organizada.
- Testar estratégias em períodos passados antes de as usar em tempo real.
- Mostrar sinais de entrada, saída ou neutralidade com explicação.
- Preparar ordens com todos os detalhes visíveis.
- Exigir confirmação manual por ordem.
- Começar sempre em paper trading, ou seja, simulação com dinheiro fictício.
- Guardar um histórico completo para auditoria e aprendizagem.
- Exportar operações realizadas em formato útil para IRS em Portugal.

Não-objetivos

- Não é uma máquina de fazer dinheiro.
- Não é aconselhamento financeiro.
- Não deve executar ordens sem confirmação humana.
- Não deve ter botão de confirmar tudo.
- Não deve guardar credenciais da IBKR no código.
- Não deve começar em modo live.
- Não deve esconder custos, comissões, spreads ou slippage.
- Não deve mostrar backtests bonitos sem avisos de risco e overfitting.

3. Conceitos base explicados sem jargão

Conceito	Explicação simples
Sinal	Uma sugestão gerada por uma estratégia. Exemplo: talvez comprar AAPL agora. Um sinal não mexe em dinheiro.
Ordem preparada	Um formulário preenchido pela app com símbolo, quantidade, lado, tipo de ordem, preço e stop. Ainda não foi enviado.
Ordem enviada	A ordem que já foi transmitida para a corretora depois de confirmação manual.
Paper trading	Simulação em tempo real com dinheiro fictício. Serve para testar

	sem arriscar capital real.
Backtesting	Teste de uma estratégia em dados históricos. A pergunta é: se eu tivesse usado estas regras no passado, o que teria acontecido?
Slippage	Diferença entre o preço esperado e o preço real de execução. Em day trading, esta diferença pode estragar uma estratégia aparentemente boa.
Spread	Diferença entre o preço a que alguém quer comprar e o preço a que alguém quer vender.
Drawdown	Queda desde um ponto alto da carteira até um ponto baixo. Ajuda a perceber quanto se pode sofrer durante uma sequência má.
Overfitting	Quando se ajusta uma estratégia tão bem ao passado que ela parece brilhante no teste, mas depois falha no futuro.

Ideia central

Sinal não é ordem. Ordem preparada não é ordem enviada. Envio só acontece depois de confirmação manual.

4. Tecnologia: o que faz cada peça de software

Esta secção explica as peças técnicas como se fossem departamentos de uma pequena empresa. Cada peça tem uma função concreta e deve ter limites claros.

Peça	O que é	Para que serve neste projeto
Python	Linguagem principal do backend.	Faz análise de dados, indicadores, estratégias, backtesting, risco e ligação à IBKR.
FastAPI	Servidor/API local do backend.	Permite ao frontend pedir dados, sinais, backtests e preparação de ordens.
uvicorn	Motor que põe o FastAPI a correr.	Arranca o backend localmente.
pandas	Ferramenta para tabelas de dados.	Organiza candles, preços, volume e cálculos de indicadores.
pandas-ta / TA-Lib	Bibliotecas de indicadores técnicos.	Podem calcular RSI, MACD, Bollinger Bands, ATR, médias móveis e outros indicadores.
PostgreSQL	Base de dados principal.	Guarda candles, sinais, backtests, ordens, fills, limites de risco e auditoria.
SQLAlchemy	Camada para falar com a base de dados.	Permite ao Python gravar e ler dados sem espalhar SQL por toda a app.
Alembic	Gestor de alterações da base de dados.	Controla versões da estrutura: novas tabelas, colunas e índices.
Docker Compose	Ferramenta para correr serviços locais.	Arranca PostgreSQL e outros serviços de forma repetível.
pydantic	Validação de dados.	Garante que sinais, ordens e configurações têm formato correto.
structlog	Logs estruturados.	Regista eventos importantes de forma pesquisável e organizada.
React	Biblioteca para interface visual.	Constrói o dashboard, gráficos, botões e modais.
TypeScript	JavaScript com regras mais fortes.	Reduz erros no frontend ao validar tipos de dados.
Vite	Ferramenta de desenvolvimento frontend.	Permite correr e desenvolver a interface

		rapidamente.
TradingView lightweight-charts	Biblioteca de gráficos.	Mostra velas, linhas, indicadores e marcações de compra/venda.
ib_async	Biblioteca Python para IBKR.	Ajuda o backend a falar com o IB Gateway/TWS API.

Arquitetura simples

A arquitetura recomendada mantém o frontend longe de decisões críticas. O frontend mostra informação e pede ações. O backend valida, aplica risco, grava auditoria e só depois fala com a corretora.

Frontend React -> FastAPI Backend -> RiskEngine -> PostgreSQL / IBKRAdapter -> IB Gateway -> IBKR

Regra técnica importante

O frontend nunca deve escrever diretamente em tabelas críticas como ordens, confirmações, fills, limites de risco ou audit log. Tudo deve passar pelo backend.

5. Integração: a que sistemas a app se liga

Interactive Brokers / IBKR

A IBKR é a corretora. É o sistema onde existem a conta paper, a conta live, os dados de mercado, as posições, as ordens e as execuções. A app deve usar apenas a API oficial da corretora, nunca automação de cliques no site.

IB Gateway

O IB Gateway é um programa da Interactive Brokers que fica aberto no computador. O utilizador faz login nele. Depois a app liga-se a esse programa por uma porta local. Isto evita guardar a password da IBKR no código.

App local -> IB Gateway autenticado -> Interactive Brokers

TWS

TWS significa Trader Workstation. É a aplicação completa de trading da Interactive Brokers. Também permite ligação via API, mas para este projeto o IB Gateway é normalmente preferível por ser mais leve e mais adequado a uma app local.

IBKR API

A API da IBKR é a porta oficial para o programa pedir dados e, mais tarde, enviar ordens confirmadas. Serve para consultar preços, histórico, posições, saldo, ordens abertas, fills e estado da ligação.

IBKRAdapter

O IBKRAdapter é o tradutor entre a app e a IBKR. O resto do sistema pede coisas simples como "dá-me candles" ou "envia esta ordem confirmada". O adapter traduz isso para a linguagem da IBKR.

Porque usar um adapter?

Para a app não ficar toda misturada com detalhes da IBKR. Se um dia for preciso trocar ou adicionar outra corretora, a alteração fica mais controlada.

Fontes de dados históricos

No início, os dados históricos podem vir de ficheiros CSV ou de outra fonte controlada. Isto permite construir o motor de análise e backtesting antes de perder tempo com detalhes da corretora.

Taxas de câmbio para EUR

Para exportação IRS, as operações em USD ou noutra moeda precisam de conversão para EUR. A fonte da taxa de câmbio deve ser explícita e a taxa usada deve ficar guardada por linha.

6. Dados e PostgreSQL: estrutura recomendada

A decisão recomendada é usar PostgreSQL, localmente, através de Docker Compose. Isto dá uma estrutura mais séria do que SQLite sem obrigar a colocar dados sensíveis na cloud logo no arranque.

Decisão assumida

Usar PostgreSQL + SQLAlchemy + Alembic desde o início. Supabase pode ser avaliado mais tarde, mas não deve ser a base crítica inicial.

Tabela	Função
instruments	Lista de ativos: AAPL, TSLA, ETFs, Forex, etc.
market_bars	Velas históricas e atuais: open, high, low, close, volume, timeframe e timestamp.
indicator_values	Indicadores calculados, se fizer sentido guardar em vez de recalcular.
strategies	Estratégias disponíveis.
strategy_versions	Versões das estratégias. Importante para saber que regras geraram cada sinal.
signals	Sinais gerados: BUY, SELL, FLAT, força, racional e momento.
backtest_runs	Cada simulação feita, com parâmetros, período, capital inicial, custos e slippage.
backtest_trades	Trades simulados dentro de cada backtest.
backtest_metrics	Métricas do backtest: retorno, drawdown, win rate, profit factor, Sharpe, etc.
prepared_orders	Ordens preparadas mas ainda não enviadas.
order_confirmations	Registo das confirmações ou rejeições manuais.
broker_orders	Ordens enviadas para a IBKR, em paper ou live.
fills	Execuções devolvidas pela corretora.
positions_snapshots	Fotografias das posições ao longo do tempo.
risk_limits	Limites configurados: tamanho máximo, perda diária, número de posições.
audit_log	Registo imutável de eventos importantes.
fx_rates	Taxas de câmbio usadas para conversão para EUR.
tax_exports	Histórico de exportações para IRS.

Supabase pode fazer sentido numa fase futura se houver necessidade de autenticação, cloud, backups geridos, realtime ou acesso por vários dispositivos. Ainda assim, as operações críticas devem continuar a passar pelo backend.

7. Segurança, risco e auditoria

Esta app tem de ser desenhada com travões desde o primeiro dia. O perigo não é só técnico; é financeiro. Um erro pequeno pode preparar uma ordem errada, usar quantidade errada ou ignorar uma perda.

Proteção	O que faz
Paper por defeito	A app começa sempre em modo paper. Live é uma decisão separada e deliberada.
Confirmação por ordem	Cada ordem exige confirmação manual específica. Nada de confirmar em lote.
Sem execução automática	A app não pode ter modo robô autónomo nesta fase.
RiskEngine	Valida limites antes de uma ordem aparecer como confirmável.
Kill switch	Cancela ordens abertas e bloqueia novas preparações.
Credenciais fora do código	A password da IBKR nunca é guardada pela app. O login acontece no IB Gateway/TWS.
Audit trail	Cada sinal, ordem, confirmação, rejeição e fill fica registado.
Banner de modo	O utilizador vê sempre se está em PAPER ou LIVE.

Travão essencial

Uma ordem só deve chegar à IBKR pelo caminho oficial: preparação -> validação de risco -> revisão visível -> confirmação manual -> envio.

8. Plano de construção por fases

A ordem das fases foi ajustada para dar prioridade ao motor de análise e simulação. A ligação à IBKR vem depois de o projeto já conseguir testar estratégias com dados históricos.

Fase 1 - Scaffold e base técnica

Criar o repositório, backend FastAPI, frontend React, configuração, Docker Compose com PostgreSQL, logging, estrutura de pastas e banner de modo PAPER.

Fase 2 - Dados históricos e modelo de mercado

Criar tabelas e importador de dados OHLCV. Permitir carregar candles de CSV ou outra fonte controlada.

Objetivo: ter dados fiáveis para análise.

Fase 3 - IndicatorEngine

Calcular indicadores como médias móveis, RSI, MACD, Bollinger Bands, ATR, VWAP e volume. Os cálculos devem ser testáveis e reutilizáveis.

Fase 4 - StrategyEngine e sinais

Criar estratégias simples que leem dados e indicadores e devolvem sinais. Exemplo: RSI mean reversion, MACD crossover, cruzamento de médias.

Fase 5 - Backtesting primeiro

Correr estratégias em períodos passados. Mostrar trades simulados, curva de capital, drawdown, win rate, profit factor, custos, slippage e comparação com buy-and-hold.

Fase 6 - Dashboard de análise

Mostrar gráficos, indicadores, sinais e resultados de backtests numa interface clara. Cada sinal deve ter explicação.

Fase 7 - IBKRAdapter em paper

Ligar ao IB Gateway em modo paper para consultar conta, posições, dados históricos e, depois, dados em tempo real ou atrasados.

Fase 8 - RiskEngine e kill switch

Aplicar limites de risco no backend: tamanho máximo por ordem, exposição, número de posições, perda diária, stop obrigatório e botão de bloqueio.

Fase 9 - Preparação e confirmação de ordens em paper

Permitir preparar uma ordem a partir de um sinal ou manualmente. A ordem passa pelo RiskEngine, é mostrada ao utilizador e só é enviada em paper após confirmação manual.

Fase 10 - Exportação IRS

Gerar CSV/XLSX com operações realizadas, com valores em EUR, comissões, datas, preços, P&L e fonte de câmbio usada.

Fase 11 - Guardrails de live

Construir os travões para um eventual modo live: fricção extra, reconhecimento explícito, banner persistente e impossibilidade de trocar de paper para live com um clique simples.

9. Fluxos práticos de utilização

Fluxo A - Testar uma estratégia no passado

9. Escolher símbolo, por exemplo AAPL.
10. Escolher período, por exemplo janeiro a junho.
11. Escolher timeframe, por exemplo candles de 5 minutos.
12. Escolher estratégia e parâmetros.
13. Definir capital inicial, comissões e slippage.
14. Correr o backtest.
15. Analisar resultados: lucro, perdas, drawdown, número de trades e estabilidade.

Fluxo B - Receber um sinal

16. A app recebe dados novos.
17. Atualiza indicadores.
18. A estratégia gera um sinal.
19. O dashboard mostra o sinal com motivo.
20. O utilizador decide ignorar, estudar ou preparar ordem.

Fluxo C - Preparar uma ordem paper

21. O utilizador escolhe agir sobre um sinal.
22. A app preenche símbolo, lado, quantidade, tipo de ordem e preço.
23. O RiskEngine valida limites.
24. A app mostra a confirmação com todos os detalhes.
25. O utilizador confirma manualmente.
26. A ordem é enviada para paper trading.
27. A app acompanha estado, fills e resultado.
28. Tudo fica no audit log.

Fluxo D - Um dia, talvez live

Live não é objetivo inicial. Só deve ser considerado depois de backtests sólidos, paper trading prolongado, auditoria consistente, limites de risco testados e plena consciência de que dinheiro real está em risco.

Anexo - frase de orientação para a equipa

Frase para manter visível

Esta app existe para informar decisões, testar ideias e obrigar a confirmação consciente. Não é, e não deve tornar-se, um trader autónomo.

Resumo final: construir primeiro o motor de análise e simulação histórica; só depois ligar à IBKR em paper; e só muito mais tarde considerar qualquer aproximação ao modo live.